Danny Rehl, Programmieren I - Python

Übungsblatt 02

Abgabefrist: Montag, den 02.11.2026, um 23:59 Uhr

Wichtig: Sie finden für dieses Übungsblatt eine Vorlage für Ihre Abgabe zum Herunterladen im Moodle.

Aufgabe 1 (1½ Punkte)

Weisen Sie den Variablen `A` bis `D` passende Integer-Werte zu, sodass die folgenden Ausgaben entstehen:

```
>>> print(A + 2 * B)
35671
>>> print((3 ** (C - 1)) + 1)
19684
>>> print(True and not D - 37)
True
```

Hinweis: Beachten Sie Operator-Präzedenz. Also zum Beispiel:

```
>>> 1 + 2 * 3
7
>>> (1 + 2) * 3
9
```

Aufgabe 2 (2 Punkte)

Das folgende Zitat aus dem Wikipedia-Artikel zu Python ist in der Variablen `QUOTE` gespeichert:

> Python ist eine universelle, üblicherweise interpretierte höhere Programmiersprache.

Weisen Sie den Variablen `F` bis `N` passende Integer-Werte zu, wobei `I`, `K` und `L` *negative* Integer-Werte sein sollen, sodass nach Anwenden von Indexing und Slicing auf den String folgende Ausgaben entstehen:

```
>>> print(QUOTE[F:G] + QUOTE[H] + QUOTE[I] + 2 * QUOTE[J] + QUOTE[K])
Python ist toll.
>>> print(QUOTE[:L] + "su" + QUOTE[48:M:2] + QUOTE[-6::N])
Python ist super.
```

Aufgabe 3 (1 Punkt)

Sie haben in der Vorlesung Strings und die Methoden dieses Datentyps kennengelernt. Wir wollen dieses Wissen benutzen, um in den folgenden Aufgaben eine erste rudimentäre Korpusanalyse durchzuführen.

Die Variable `CORPUS_TEXT` enthält unseren zu analysierenden Beispieltext. Sie finden in der Vorlage bereits mehrere Beispiele und können auch die Beispiele aus der Aufgabenstellung oder Ihre eigenen Beispiele zum Testen verwenden. Wichtig ist nur, dass Ihr Code allgemein formuliert ist und am Ende mit *allen* diesen Texten funktioniert und die richtigen Ergebnisse produziert.

Wir verwenden für die Aufgaben vereinfachte Definitionen. Zunächst wollen wir herausfinden, wie viele *Sätze* das Korpus enthält.

Definition: Ein *Satz* ist ein String, der mit dem Punktzeichen `.` endet. Sie können davon ausgehen, dass keine anderen Satzzeichen, wie `!` oder `?`, ein Satzende markieren. Ihr Code muss also nur Punkte unterstützen. Gleichzeitig können Sie andererseits annehmen, dass alle Punkte im Text auch Satzende sind, und nicht beispielsweise Dezimal- oder Tausendertrennzeichen.

Als ersten Schritt soll `CORPUS_TEXT` in seine Sätze aufgeteilt und das Ergebnis in einer Liste `SENTENCES` gespeichert werden. Jedes der Elemente der Variable `SENTENCES` enthält dann einen der Sätze, also entspricht die Anzahl der Elemente in `SENTENCES` genau der Anzahl der Sätze im Korpus. Diese Zahl soll dann in `SENTENCES_COUNT` gespeichert werden.

Es gibt mehrere korrekte Varianten für `SENTENCES`, denn zwischen den Sätzen befindet sich ein Punkt und ein Leerzeichen. Je nachdem, was man genau noch als Bestandteil eines Satzes definiert, erhält man unterschiedliche Ergebnisse für die Elemente in `SENTENCES`. Das spielt für diese Aufgabe allerdings keine Rolle, denn für die gesuchte *Anzahl* der Sätze ist diese Information irrelevant.

Beispiel:

```
>>> CORPUS_TEXT
'I have no quarrel with you, good Sir Knight. But I must cross this bridge.'
```

Mögliche Lösungen für `SENTENCES`:

```
>>> SENTENCES
['I have no quarrel with you, good Sir Knight', 'But I must cross this bridge.']
```

Oder:

```
>>> SENTENCES
['I have no quarrel with you, good Sir Knight', ' But I must cross this bridge']
```

Oder:

```
>>> SENTENCES
['I have no quarrel with you, good Sir Knight.', ' But I must cross this bridge.']
```

↪

Alle diese Lösungen liefern das korrekt Endergebnis für `SENTENCES_COUNT`:

```
>>> SENTENCES_COUNT
2
```

Tipp: Zum Trennen eines Strings (`CORPUS_TEXT`) in eine Liste mehrerer Substrings (`SENTENCES`) gibt es eine passende String-Methode und zum Zählen der Elemente einer Liste (für `SENTENCES_COUNT`) gibt es eine passende Built-in-Funktion. Ganz allgemein werden sich die Methoden von Variablen vom Typ String für die folgenden Aufgaben als äußerst nützlich erweisen. Es lohnt sich also, alle einfach mal anzuschauen, um einen Überblick zu bekommen.

Außerdem kann es sinnvoll sein, mehrere `print`-Befehle zum "Debugging" einzubauen, also die Werte der verwendeten Variablen in Zwischenschritten anzuschauen. Sie können dann diese Ausgabe Ihres Codes damit vergleichen, was Sie "manuell" durch Untersuchen des jeweiligen `CORPUS_TEXT` (durch Zählen der Sätze) als Lösung erwarten würden.

Häufiger Fehler: Bedenken Sie, dass auch der letzte Satz mit einem Punkt endet und was das eventuell für den letzten Eintrag in Ihrer Liste bedeutet, wenn Sie `str.split` verwenden. Sie sollten sich im Zweifel Ihr Zwischenergebnis mit `print(SENTENCES)` anschauen.

Aufgabe 4 (1 Punkt)

Wir möchten nun den Text tokenisieren und die Anzahl der Token im Korpus bestimmen.

Definition: Ein *Token* definieren wir als eine Reihe von Zeichen, die zwischen zwei Leerzeichen oder Satzzeichen beziehungsweise am Anfang des Strings steht. Satzzeichen sind für dieses Aufgabenblatt keine Tokens.

Für die Tokenisierung stören uns zunächst also die Satzzeichen, welche laut unserer Definition kein Token sind. Sie können davon ausgehen, dass in `CORPUS_TEXT` nur Punkt `.`, Doppelpunkt `:` und Komma `,` als Satzzeichen vorkommen und auch jedes dieser Zeichen ein Satzzeichen ist, welches entfernt werden muss. Entfernen Sie alle diese Satzzeichen aus dem Text in `CORPUS_TEXT` und speichern Sie das Ergebnis als `CORPUS_CLEANED`. Die Variable `CORPUS_CLEANED` soll ein String sein, also eine Variable vom Typ `str`.

Anschließened tokenisieren Sie den Wert in `CORPUS_CLEANED`, weisen Sie das Ergebnis der Variablen `TOKENS` zu und speichern Sie die erzeugte Anzahl der Tokens in der Variablen `TOKENS_COUNT`. Es soll also `TOKENS` eine Variable vom Typ `list` und `TOKENS_COUNT` eine Variable vom Typ `int` sein.

Beispiel:

```
>>> CORPUS_TEXT
"Well, on second thought, let's not go to Camelot: It is a silly place."
```

Entfernen der Satzzeichen:

```
>>> CORPUS_CLEANED
"Well on second thought let's not go to Camelot It is a silly place"
```

Tokenisierung:

```
>>> TOKENS
['Well', 'on', 'second', 'thought', "let's", 'not', 'go', 'to', 'Camelot', 'It',
'is', 'a', 'silly', 'place']
```

Anzahl der Tokens:

```
>>> TOKENS_COUNT
14
```

Tipp: “Entfernen” ist hier äquivalent zu “durch einen leeren String ersetzen”.

Erinnerung: Alle diese Aufgaben lassen sich mit String-Methoden lösen.

Aufgabe 5 (1 Punkt)

Wir wollen nun Type von Token unterscheiden.

Definition: Einen *Type* definieren wir als ein Wort, welches in seiner Form zwar mehrfach vorkommen darf, aber nur einmal gezählt wird. Bei unserer Definition soll Groß- und Kleinschreibung *nicht* vernachlässigt werden.

Bestimmen Sie aus den Tokens in `TOKENS` die Types sowie deren Anzahl und speichern Sie das Ergebnis in den Variablen `TYPES` beziehungsweise `TYPES_COUNT` ab. Dabei ist `TYPES` eine aufzählbare (iterable) Variable und `TYPES_COUNT` eine Variable vom Typ `int`.

Beispiel:

```
>>> CORPUS_TEXT
'Be quiet. I order you to be quiet.'
```

Tokenisierung aus vorheriger Aufgabe:

```
>>> TOKENS
['Be', 'quiet', 'I', 'order', 'you', 'to', 'be', 'quiet']
```

Types:

```
>>> TYPES
{'Be', 'quiet', 'I', 'order', 'you', 'to', 'be'}
```

Beachte: `'Be'` und `'be'` sind verschiedene Types (einmal mit Groß-, einmal mit Kleinbuchstaben beginnend) und das doppelt vorkommende Token `'quiet'` zählt nicht mehrfach, sondern es gibt nur einen Type `'quiet'`.

Anzahl der Types:

```
>>> TYPES_COUNT
7
```

Tipp: Der geforderte Datentyp von `TYPES` ist absichtlich so frei gehalten. Sie müssen hier keine Liste verwenden, sondern haben einen anderen aufzählbaren Datentyp kennengelernt, welcher für `TYPES` besonders geeignet ist. Die Reihenfolge der Types in `TYPES` spielt keine Rolle.

Aufgabe 6 (1 Punkt)

Wir interessieren uns nun für denjenigen Type in `TYPES`, welcher die größte Wortlänge aufweist.

Es gibt mehrere Möglichkeiten, den längsten Type zu finden. Für diese Art von Problem bieten sich beispielsweise die Built-in Functions `min`, `max` oder `sorted` an, welche alle die Option besitzen, ein extra `key`-Argument zu verwenden, welches die "Metrik" der Sortierung festlegt. Um die Wortlänge zu bestimmen können Sie `len` benutzen.

Am Beispiel der Types in `TYPES` aus vorheriger Aufgabe:

```
>>> TYPES
{'Be', 'quiet', 'I', 'order', 'you', 'to', 'be'}
```

Beispielaufruf von `sorted` mit Argument `key=len`:

```
>>> sorted(TYPES, key=len)
['I', 'be', 'to', 'Be', 'you', 'order', 'quiet']
```

Eine Lösung wäre also:

```
>>> LONGEST_TYPE
'quiet'
```

Machen Sie sich durch weitere, eigene Beispiele und Lesen der Python-Dokumentation klar, wie das `key`-Argument funktioniert und wie Sie aus dem Ergebnis einen passenden Wert für `LONGEST_TYPE` erhalten können.

Hinweis: Falls es mehrere Types mit derselben, maximalen Wortlänge gibt, dann spielt es keine Rolle, welcher Type ausgewählt wird. Alle Types der maximalen Wortlänge sind korrekte Lösungen.

Für das Beispiel oben gibt es also noch eine weitere, alternative Lösung:

```
>>> LONGEST_TYPE
'order'
```

Aufgabe 7 (2½ Punkte)

Abschließend sollen ein paar Eigenschaften von `LONGEST_TYPE` bestimmt werden.

Erstellen Sie eine Variable `LONGEST_TYPE_PROPERTIES` vom Typ `dict` mit folgendem Inhalt:

- Konvertieren Sie alle Zeichen von `LONGEST_TYPE` zu Großbuchstaben und speichern Sie den `str`-Wert unter dem Schlüssel `'letters_converted_to_capitals'`.
- Speichern Sie einen `bool`-Wert unter dem Schlüssel `'contains_only_letters'`, welcher angibt, ob `LONGEST_TYPE` nur aus Buchstaben besteht.
- Falls alle Buchstaben in `LONGEST_TYPE` Kleinbuchstaben sind, soll der Schlüssel `'all_letters_are_lowercase'` den `bool`-Wert `True` enthalten, ansonsten `False`.
- Wie oft kommt der Kleinbuchstabe `e` in `LONGEST_TYPE` vor? Speichern Sie das Ergebnis als `int`-Wert unter dem Schlüssel `'how_many_letter_e'`.
- Der Schlüssel `'index_of_first_letter_e'` soll den Index (als `int`-Wert) des ersten Vorkommens des Kleinbuchstabens `e` in `LONGEST_TYPE` enthalten, beziehungsweise `-1`, falls `LONGEST_TYPE` kein `e` enthält.

Ein Beispielwert für `LONGEST_TYPE`:

```
>>> LONGEST_TYPE
'Knight'
```

Und die dazu passenden Werte der Lösung für `LONGEST_TYPE_PROPERTIES`:

```
>>> print(repr(LONGEST_TYPE_PROPERTIES).replace(',', ',\n'))
{'letters_converted_to_capitals': 'KNIGHT',
 'contains_only_letters': True,
 'all_letters_are_lowercase': False,
 'how_many_letter_e': 0,
 'index_of_first_letter_e': -1}
```

Ein weiteres, alternatives Beispiel:

```
>>> LONGEST_TYPE
'bridge-keeper'
```

Erwartete Werte der Lösung für `LONGEST_TYPE_PROPERTIES`:

```
>>> print(repr(LONGEST_TYPE_PROPERTIES).replace(',', ',\n'))
{'letters_converted_to_capitals': 'BRIDGE-KEEPER',
 'contains_only_letters': False,
 'all_letters_are_lowercase': True,
 'how_many_letter_e': 4,
 'index_of_first_letter_e': 5}
```

Erinnerung: Alle diese Aufgaben lassen sich mit String-Methoden lösen.

Zusatzaufgabe 8 (keine Punkte)

Wir möchten den fünften Buchstaben des Worts “Hallo” ausgeben. Dazu weisen wir das Wort als String der Variablen `S` mittels `S = 'Hallo'` zu. Da “Hallo” fünf Buchstaben hat, schreiben wir `print(S[5])` zum Ausgeben des fünften Buchstabens. Statt des gewünschten Buchstabens “o” erhalten wir jedoch folgenden Fehler:

```
>>> S = 'Hallo'
>>> print(S[5])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

Was haben wir falsch gemacht? Es gibt eine Eigenschaft von Sequences in Python, welche dafür verantwortlich ist.

Sie können Ihre Ideen hierzu als Kommentar in der Abgabedatei notieren. Es wird allerdings keine automatische Rückmeldung von Tutron geben, sondern Sie können die Aufgabe bei Bedarf gemeinsam im Tutorium diskutieren.

Zusatzaufgabe 9 (keine Punkte)

Wir möchten jeweils den letzten Eintrag in unseren beiden Variablen `T = [1, 2, 3]` und `U = (1, 2, 3)` von `3` auf `4` ändern.

Bei `T` funktioniert das auch:

```
>>> T = [1, 2, 3]
>>> print(T[-1])
3
>>> T[-1] = 4
>>> print(T[-1])
4
```

Bei `U` erscheint allerdings folgender Fehler:

```
>>> U = (1, 2, 3)
>>> print(U[-1])
3
>>> U[-1] = 4
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

Anscheinend können wir bei Variablen vom Typ `tuple` keine Einträge ändern. Was ist der Grund hierfür? (Es gibt einen speziellen Begriff für diese Eigenschaft von bestimmten Datentypen in Python.)

Sie können Ihre Ideen hierzu als Kommentar in der Abgabedatei notieren. Es wird allerdings keine automatische Rückmeldung von Tutron geben, sondern Sie können die Aufgabe bei Bedarf gemeinsam im Tutorium diskutieren.